جزوه آموزشی Jakarta EE در پروژه فروشگاه

معماری و Mapper و Cache و Security

تهیه و تدوین احمد مصباح

آموزش گام به گام با مثال‌های پروژه MftOnlineStoreEE

JDK 17 · Jakarta EE 10 · TomEE · PostgreSQL · Redis

نسخه آموزشی ۱ · بررسی کد در ۲۰۲۶/۱۰/۱۳

این جزوه برای دانشجویی نوشته شده است که Java، کلاس، record ،interface و SQL مقدماتی را می‌شناسد و می‌خواهد یک ماژول واقعی را در پروژه بنویسد. هدف، فهم ارتباط اجزاست؛ نه حفظ annotationها یا ساخت معماری پیچیده.

اصل دوره: هر مسئولیت یک جای مشخص داشته باشد. زیرساخت مشترک را دوباره ننویسیم؛ منطق کسب‌وکار را در ماژول خودش نگه داریم.

راهنمای مطالعه و فهرست

چگونه از جزوه استفاده کنیم

هر بخش را به این ترتیب بخوانید: مفهوم، فایل متناظر در پروژه، مثال و تمرین. قبل از اضافه‌کردن امکانات، مسیر یک درخواست را در کد دنبال کنید.

سه برچسب مهم: «کد پروژه» یعنی مثال از فایل موجود است. «نمونه آموزشی» یعنی مثال برای توضیح ساده شده است. «کار باقی‌مانده» یعنی قابلیت هنوز آماده یا تأییدشده نیست.

موضوع	صفحه
معماری و مرز ماژول‌ها	۳ و ۴
CDI و تزریق وابستگی	۵
Entity و Repository	۶ و ۷
DTO و Validation	۸
Mapper و MapStruct	۹ و ۱۰
Transaction و Lazy	۱۱
REST و پاسخ مشترک	۱۲
Cache و Redis	۱۳ تا ۱۶
Security و BCrypt	۱۷ تا ۲۰
Attachment و فایل	۲۱
OpenAPI	۲۲
UI و fetch مشترک	۲۳
Docker و تنظیمات	۲۴
تست و عیب‌یابی	۲۵
ساخت ماژول دانشجویی	۲۶
پرسش‌های مرور و پاسخ	۲۷
واژه‌نامه و منابع	۲۸

محدوده آموزش: پروژهٔ محلی و آموزشی است. بخش امنیت کامل نیست؛ نمونه‌های این فصل نباید به‌عنوان امنیت فعال برنامه معرفی شوند. تست و build نیز با اجرای واقعی سرور متفاوت‌اند.

۱ معماری لایه‌ای و مسیر درخواست

چرا لایه‌ها را جدا می‌کنیم

فرض کنید کاربر می‌خواهد یک دپارتمان ایجاد کند. API باید ورودی HTTP را دریافت کند؛ service باید وجود و فعال‌بودن شعبه و تکراری‌نبودن کد را بررسی کند؛ repository باید داده را بخواند و ذخیره کند. mapper فقط شکل داده را تبدیل می‌کند.

اگر همهٔ این کارها داخل API نوشته شود، منطق به HTTP وابسته می‌شود و تست یا استفادهٔ دوباره سخت خواهد شد.

```
Browser
   |
   v
REST API --> Request DTO
   |
   v
Service --> Business rules --> Mapper --> Response DTO
   |
   v
Repository --> EntityManager --> PostgreSQL
```

مسئولیت هر لایه

لایه	انجام می‌دهد	نباید انجام دهد
REST	مسیر، validation، method و status	محاسبه مبلغ یا query دیتابیس
Service	قواعد، transaction و بارگذاری رابطه	ساخت HTML یا مدیریت هدر HTTP
Repository	query و persist/merge/remove	تصمیم درباره مجوز یا وضعیت سفارش
Mapper	تبدیل داده و نام فیلدها	اجرای query و احراز هویت
Entity	مدل پایدار و رابطه‌های JPA	دریافت request یا ساخت response

نمونه پروژه: در Department.create ابتدا کد بررسی و Branch بارگذاری می‌شود؛ سپس request به entity تبدیل، رابطه تعیین و DTO خروجی ساخته می‌شود.

تمرین: مسیر POST یک Attachment را از API تا repository دنبال کنید. بنویسید پاسخ 201 کجا ساخته می‌شود و تکراری‌بودن نام فایل کجا بررسی می‌شود.

فایل مطالعه: `organization/service/impl/DepartmentServiceImpl.java` و `attachments/rest/AttachmentApi.java`.

۲ مرز ماژول‌ها و زیرساخت مشترک

ساختار پروژه

ماژول یعنی مجموعهٔ کدهای مربوط به یک مسئولیت کسب‌وکار. داخل هر ماژول پوشه‌های entity، repository، dto، rest، service، mapper و exception قرار می‌گیرند.

```
com.mftplus.mftonlinestoreee
  core / config / integration
  attachments
  users / products / organization
  orders / inventories / tickets
  delivery / financials
```

بخش	مسئولیت
core	قراردادهای پایه، پاسخ، خطا، paging و logging
config و integration	datasource، EntityManager و اتصال TomEE به Hibernate
attachments	مشخصات و محتوای پیوست
users	کاربر، نقش و پروفایل
products	محصول و دسته‌بندی
organization	سازمان، شعبه، دپارتمان و عضویت کارمند
orders و inventories	سفارش و گردش موجودی
tickets و delivery	پشتیبانی و مرسوله
financials	بانک، حساب، صندوق و پرداخت

چگونه وابستگی را کنترل کنیم

- ماژول به Core وابسته است؛ Core نباید برای هر ماژول import جدید بگیرد.
- service می‌تواند repository ماژول مرتبط را برای بارگذاری entity استفاده کند.
- تغییر User یا Product باید با گروه‌های مصرف‌کننده هماهنگ شود.
- Attachment با ownerType/ownerId به مالک اشاره می‌کند؛ برای هر مالک service جدا ندارد.
- هر منطق مشترک را فوراً به Core منتقل نکنید؛ اول ببینید واقعاً چند مصرف‌کننده دارد.

تقسیم کار دوره: زیرساخت و Attachment با مدرس است؛ دانشجو ماژول خودش را با قراردادهای آماده تکمیل می‌کند. abstraction تازه فقط برای نیاز واقعی ساخته شود.

تمرین: وابستگی دپارتمان به User و Branch را رسم کنید. توضیح دهید چرا User لازم نیست فهرست دپارتمان‌هایش را نگه دارد.

۳ CDI و تزریق وابستگی

مدیریت شیء با Container

CDI شیءهای برنامه را پیدا و وابستگی‌هایشان را تزریق می‌کند. وقتی service را با `new` بسازیم، انتظار تزریق خودکار، transaction یا logging از Container نداریم.

کد پروژه به‌صورت کوتاه:

```
@ApplicationScoped
public class DepartmentDtoLoader {
    @Inject
    private DepartmentRepository repository;

    @Inject
    private DepartmentMapper mapper;

    @Transactional
    public Optional<DepartmentDto> findById(Long id) {
        return repository.findById(id).map(mapper::toDto);
    }
}
```

annotation	مفهوم در پروژه
@Inject	وابستگی از Container دریافت می‌شود
@ApplicationScoped	bean مشترک در طول عمر application
@RequestScoped	bean با عمر یک request
@Produces	معرفی منبع تولید یک وابستگی
@PostConstruct	آماده‌سازی پس از تزریق
@PreDestroy	آزادسازی منابع هنگام پایان عمر bean

نکته‌های عملی

در beans.xml حالت کشف annotated است؛ کلاس‌های سرویس و repository باید scope مناسب داشته باشند. API معمولاً RequestScoped است و mapper با تنظیم JAKARTA_CDI تولید می‌شود.

ApplicationScoped به معنی thread-safe بودن همهٔ فیلدها نیست. اطلاعات request، رمز کاربر یا entity یک درخواست را در فیلد مشترک service نگه ندارید.

Interceptor: annotationهایی مثل Transactional و Loggable هنگام فراخوانی از مسیر مدیریتی Container عمل می‌کنند. فراخوانی داخلی همان شیء در این پروژه را جای ورود از bean دیگر فرض نکنید.

تمرین: تفاوت inject کردن DepartmentDtoLoader با ساخت آن توسط new را توضیح دهید. اگر یک dependency دو implementation داشته باشد، چرا انتخاب آن ممکن است مبهم شود؟

منبع تکمیلی: مفاهیم CDI.

۴ Entity و چرخه عمر داده

Entity با DTO تفاوت دارد

Entity مدل database است و می‌تواند رابطه، شناسه، version و وضعیت حذف داشته باشد. BaseEntity یک MappedSuperclass است؛ جدول جدا ندارد و فیلدهایش در entity فرزند قرار می‌گیرند.

فیلد پایه	کاربرد
id	شناسهٔ ساخته‌شده توسط database
version	optimistic locking و تشخیص تغییر همزمان
deleted	حذف نرم
createdAt	زمان ایجاد در PrePersist
updatedAt	زمان تغییر در PreUpdate
deletedAt	زمان حذف در SQL حذف نرم

نمونه آموزشی از تنظیم entity پیوست:

```
@Entity
@Table(name = "attachments")
@SQLDelete(sql =
    "update attachments set deleted=true, " +
    "deleted_at=current_timestamp where id=? and version=?")
@SQLRestriction("deleted = false")
public class Attachment extends BaseEntity {
}
```

این مثال فقط بخش annotationها را نشان می‌دهد؛ constructor، فیلدها و Lombok در فایل اصلی هستند. SQLDelete و SQLRestriction امکانات Hibernate هستند، نه annotationهای استاندارد JPA.

سه نکته مهم

۱. شناسه و زمان‌ها را در entity فرزند دوباره تعریف نکنید. enum را با STRING ذخیره کنید تا ترتیب ثابت‌ها معنای داده را عوض نکند.

۲. حذف نرم یعنی ردیف باقی می‌ماند. constraint یکتای database ممکن است اجازه ندهد مقدار ردیف حذف‌شده دوباره استفاده شود؛ صرف بررسی service این تعارض را حل نمی‌کند.

۳. version را client تعیین نکند. دو تغییر همزمان می‌توانند به خطای optimistic lock برسند؛ «وجود version» جای تست همزمانی نیست.

تمرین: تفاوت حذف نرم و حذف فیزیکی را برای فایل پیوست توضیح دهید.

فایل مطالعه: `core/entity/BaseEntity.java` و `attachments/entity/Attachment.java`.

۵ Repository و صفحه‌بندی

query در جای درست

BaseRepository عملیات عمومی را دارد. repository ماژول، entityType و finderهای خاص را معرفی می‌کند. save از persist، update از merge و delete از remove استفاده می‌کند.

نمونه آموزشی finder با JPQL:

```
public List<Department> findByBranchId(Long branchId) {
    return entityManager.createQuery(
            "select department from Department department " +
            "where department.branch.id = :branchId " +
            "order by department.id", Department.class)
        .setParameter("branchId", branchId)
        .getResultList();
}
```

JPQL نام entity و فیلد Java را می‌شناسد؛ SQL نام جدول و ستون را. مقدار کاربر را به متن query نچسبانید؛ setParameter به‌کار ببرید.

قراردادهای آماده

متد	نتیجه
findById(id)	Optional entity؛ برای نبودن رکورد خالی
findAll()	فهرست مرتب‌شده بر اساس id
findAll(page, size)	فقط محتوای یک صفحه
findPage(page, size)	PageResult همراه تعداد کل
count()	تعداد رکوردهای قابل مشاهده
restore(id)	بازگرداندن حذف نرم با UPDATE native

```
public PageResult<DepartmentDto> findAll(int page, int size) {
    return repository.findPage(page, size).map(mapper::toDto);
}
```

صفحه‌ها از صفر شروع می‌شوند. برای page=2 و size=10، offset برابر 20 است. ترتیب ثابت برای paging ضروری است؛ این روش snapshot مشترک میان requestها ایجاد نمی‌کند.

تمرین: چرا List ده‌تایی به‌تنهایی برای محاسبه تعداد صفحه‌ها کافی نیست؟ چرا مقایسه `bank = :bankId` با id عددی می‌تواند اشتباه باشد؟

فایل مطالعه: `core/repository/BaseRepository.java` و `core/dto/PageResult.java`.

۶ DTO و Validation

قرارداد ورودی را از مدل database جدا کنیم

Request فقط فیلدهایی را می‌گیرد که client اجازه دارد تغییر دهد. DTO خروجی نیز فقط دادهٔ موردنیاز UI را منتشر می‌کند. قرار دادن entity داخل DTO می‌تواند رابطه‌های Lazy، حلقه JSON و دادهٔ حساس را به بیرون ببرد.

کد پروژه:

```
public record DepartmentRequest(
    @NotNull @Positive Long branchId,
    @NotBlank @Size(max = 150) String name,
    @NotBlank @Size(max = 50) String code,
    @Size(max = 500) String description
) {
}
```

branchId عدد است؛ client نباید یک Branch کامل ارسال کند. status دپارتمان در این request نیست و با عملیات وضعیت تغییر می‌کند.

annotation مناسب نوع داده

نوع	قاعدهٔ مناسب
String اجباری	NotBlank و Size
شناسه اجباری	NotNull و Positive
مقدار اختیاری	constraint مربوط، بدون الزام بی‌دلیل
List قلم‌ها	NotEmpty و Valid برای بررسی اعضا
ایمیل	Email همراه الزام موردنیاز
پول	BigDecimal و قواعد علامت/حدود

Size به‌تنهایی null یا رشتهٔ خالی را رد نمی‌کند. NotBlank را روی Boolean، enum یا عدد قرار ندهید.

در API: بدنه با `@NotNull @Valid` بررسی می‌شود. قواعدی مثل «شعبه باید فعال باشد» validation ساختاری نیستند؛ آن‌ها در service و با BusinessException بررسی می‌شوند.

پرسش تصمیم‌گیری

«کد دپارتمان ۸۰ کاراکتر است» خطای ورودی است. «همین کد در همین شعبه وجود دارد» خطای business است. این دو مسیر و پیام متفاوت دارند.

تمرین: یک Request برای محصول طراحی کنید که قیمت، نام و categoryId داشته باشد، اما id، version و entity دسته‌بندی نگیرد.

Mapper V و MapStruct

Mapper چه مشکلی را حل می‌کند

در هر create/update/read، داده بین Entity، Request و DTO جابه‌جا می‌شود. mapper این تبدیل تکراری را متمرکز می‌کند. MapStruct پیاده‌سازی را هنگام کامپایل تولید می‌کند؛ query یا منطق business را تولید نمی‌کند.

قرارداد موجود پروژه:

```
public interface BaseMapper<E extends BaseEntity, D, R> {
    D toDto(E entity);
    List<D> toDtoList(List<E> entities);

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "version", ignore = true)
    E toEntity(R request);

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "version", ignore = true)
    void updateEntity(R request, @MappingTarget E entity);
}
```

تنظیم مشترک:

```
@MapperConfig(
    componentModel = MappingConstants.ComponentModel.JAKARTA_CDI,
    unmappedTargetPolicy = ReportingPolicy.IGNORE
)
public interface MapStructConfig {
}
```

JAKARTA_CDI باعث می‌شود implementation تولیدشده قابل inject باشد. IGNORE یعنی بعضی فیلدهای بدون mapping، build را متوقف نمی‌کنند؛ پس build موفق، کامل‌بودن تبدیل داده را ثابت نمی‌کند.

چهار تبدیل

- toEntity برای ساخت entity از request.
- updateEntity برای تغییر همان entity موجود.
- toDto برای پاسخ.
- toDtoList برای فهرست پاسخ‌ها.

تمرین: بعد از کامپایل، AttachmentMapperImpl را در generated-sources باز کنید. ببینید کدام setter/builder فراخوانی می‌شود و کدام فیلد داخلی حفظ می‌شود.

مرجع: MapStruct 1.6.

۸ Mapping رابطه و نام متفاوت

شناسه رابطه را به DTO تبدیل کنیم

نمونه از DepartmentMapper:

```
@Mapper(config = MapStructConfig.class)
public interface DepartmentMapper
        extends BaseMapper<Department, DepartmentDto, DepartmentRequest> {

    @Mapping(target = "branchId", source = "branch.id")
    @Mapping(target = "branchName", source = "branch.name")
    DepartmentDto toDto(Department entity);

    @Mapping(target = "branch", ignore = true)
    @Mapping(target = "employees", ignore = true)
    @Mapping(target = "status", ignore = true)
    void updateEntity(
        DepartmentRequest request,
        @MappingTarget Department entity
    );
}
```

این بخش کوتاه‌شده است؛ فایل اصلی toEntity و تبدیل EmployeeDto را نیز دارد. service باید Branch معتبر را از repository بخواند و setBranch کند. mapper مسئول معتبر یا فعال‌بودن شعبه نیست.

نمونه آموزشی اصلاح نام متفاوت

در پروژه، ProductRequest نام active و entity نام isActive دارد. باید نام‌ها هماهنگ یا mapping صریح شود:

```
@Mapping(target = "isActive", source = "active")
@Mapping(target = "productCategory", ignore = true)
void updateEntity(
    ProductRequest request,
    @MappingTarget Product entity
);
```

همین نیاز در مسیر create هم باید بررسی شود. خروجی دسته‌بندی نیز active دارد، ولی entity آن isActive است.

چک‌لیست mapper

۱. نام فیلدها و getter واقعی را بررسی کنید. ۲. relation در خروجی به فیلد ساده تبدیل شود. ۳. فیلدهای server-owned از ورودی map نشوند. ۴. در override، تنظیمات لازم و inheritance را بررسی کنید. ۵. کد تولیدشده و تست حفظ id/version را بخوانید.

هشدار: mapper به getter رابطهٔ Lazy دست می‌زند. مناسب‌بودن mapping، بازبودن persistence context را تضمین نمی‌کند.

۹ Transaction و Lazy Loading

مرز یک عملیات business

در ثبت گردش انبار، ذخیرهٔ transaction و تغییر quantity باید با هم موفق یا با هم rollback شوند. Transactional مرز این عملیات را مشخص می‌کند.

نمونه آموزشی update روی entity managed:

```
@Transactional
public Optional<DepartmentDto> updateName(Long id, String name) {
    return repository.findById(id).map(department -> {
        department.setName(name);
        return mapper.toDto(department);
    });
}
```

این مثال فقط dirty checking را نشان می‌دهد؛ validation، business rule و ابطال کش برای عملیات واقعی باید انجام شوند. merge دوباره برای entity managed معمولاً لازم نیست.

Lazy یعنی بارگذاری هنگام نیاز

Department.branch و Branch.organization رابطهٔ Lazy دارند. mapper ممکن است برای ساخت branchName به getter دست بزند؛ بنابراین خواندن و mapping باید در context معتبر انجام شود.

در پروژه، DepartmentDtoLoader یک bean جداست و findById آن Transactional است. تبدیل DTO پیش از خروج از این مرز انجام می‌شود؛ service بیرونی بعداً DTO را کش می‌کند.

موضوع	قاعده
REQUIRED	پیوستن به transaction موجود یا شروع transaction
RuntimeException	پیش‌فرض علامت rollback
checked exception	پیش‌فرض الزاماً rollback نمی‌کند
dirty checking	تغییر entity managed در transaction ذخیره می‌شود
Lazy خارج از context	ممکن است هنگام خواندن relation خطا بدهد

برای رفتار متفاوت از rollbackOn/dontRollbackOn استفاده می‌شود؛ به این پروژه بدون نیاز اضافه نکنید.

تمرین: چرا نوشتن annotation فقط روی یک helper داخلی همان کلاس کافی نیست؟ چرا نمی‌خواهیم دادهٔ commitنشده در Redis قرار گیرد؟

مرجع: Jakarta Transactions.

۱۰ REST و پاسخ یکسان

قرارداد HTTP

RestApplication یک prefix مشترک `/api/v1` دارد. روی کلاس API فقط مسیر ماژول مثل `/attachments` قرار می‌گیرد؛ تکرار prefix مسیر را اشتباه می‌کند.

کد کوتاه از AttachmentApi:

```
@GET
@Path("/{id}")
@PermitAll
public AttachmentDto getAttachment(@PathParam("id") Long id) {
    return attachmentService.findById(id)
        .orElseThrow(() -> new NotFoundException(
            "Attachment " + id + " not found"));
}
```

API Optional را مستقیماً به client نمی‌دهد. service نبودن داده را با Optional خالی بیان می‌کند و API آن را به 404 تبدیل می‌کند.

نقش Core در پاسخ

ApiResponseFilter پاسخ JSON را پوشش می‌دهد؛ API پوشش مشترک را دوباره نسازد.

اتفاق	status
خواندن یا ویرایش موفق	200
ساخت موفق	201
حذف بدون body	204
ورودی نامعتبر	400
رکورد ناموجود	404
قاعدهٔ business نقض شده	409
خطای ناشناخته	500

```
{
  "success": true, "status": 200,
  "data": { "id": 1, "name": "Sales" },
  "timestamp": "2026-10-13T10:00:00Z"
}
```

این JSON نمونه است، نه دادهٔ واقعی database. فیلدهای null ممکن است حذف شوند. stream دانلود و 204 دوباره پوشش JSON نمی‌گیرند.

تمرین: چرا پاسخ 200 همراه success=false برای خطای business انتخاب خوبی نیست؟ چرا نباید exception mapper را در هر ماژول تکرار کنیم؟

۱۱ Cache و الگوی Cache Aside

کش منبع اصلی داده نیست

دیتابیس منبع اصلی است. کش یک کپی موقت برای خواندن‌های تکراری نگه می‌دارد. هدف این مثال کاهش queryهای تکراری findById است، نه کش‌کردن همهٔ عملیات پروژه.

اصطلاح	معنی
Hit	مقدار در کش وجود دارد
Miss	مقدار در کش نیست
TTL	مدت باقی‌ماندن مقدار
Invalidation	حذف مقدار قدیمی پس از تغییر
Cache Aside	service خودش خواندن و پرکردن کش را مدیریت می‌کند

```
findById(id)
    |
    v
GET Redis ------ hit ------> DTO
    |
   miss
    |
    v
Read database --> Map DTO --> SET with TTL --> DTO
```

انتخاب سادهٔ پروژه

در RedisDtoCache، DTO به JSON تبدیل می‌شود؛ entity و proxy JPA در Redis ذخیره نمی‌شوند. فقط سازمان و دپارتمان، با TTL برابر 300 ثانیه کش دارند. membership، page، list و رکورد ناموجود کش نمی‌شوند.

Caffeine کش داخل همان JVM است؛ Redis خارج از process است و اتصال شبکه می‌خواهد. در نسخهٔ فعلی پروژه از Redis استفاده می‌شود، نه ترکیب این دو.

کلیدها: `mft-store:cache:organization:1` و `mft-store:cache:department:1` .

تمرین: سه GET برای یک id موجود اجرا کنید. انتظار دارید کدام درخواست database را بخواند؟ اگر کلید منقضی شود چه اتفاقی می‌افتد؟

مرجع: الگوی Cache Aside در Redis.

۱۲ خواندن کش در کد پروژه

از service شروع کنیم

کد پروژه از DepartmentServiceImpl:

```
public Optional<DepartmentDto> findById(Long id) {
    if (cache.inTransaction()) {
        return loader.findById(id);
    }

    Optional<DepartmentDto> cached = cache.get(
        CacheInvalidation.Region.DEPARTMENT,
        id,
        DepartmentDto.class
    );
    if (cached.isPresent()) {
        return cached;
    }

    Optional<DepartmentDto> result = loader.findById(id);
    result.ifPresent(dto -> cache.put(
        CacheInvalidation.Region.DEPARTMENT, id, dto));
    return result;
}
```

چرا transaction موجود از کش عبور می‌کند

ممکن است همین transaction قبلاً entity را تغییر داده و هنوز commit نکرده باشد. خواندن مستقیم database/context همان عملیات، از انتشار دادهٔ نهایی‌نشده در Redis جلوگیری می‌کند. این تصمیم جای تضمین کامل هم‌زمانی را نمی‌گیرد.

عملیات سادهٔ Redis

```
GET mft-store:cache:department:1
SET mft-store:cache:department:1 "{...}" EX 300
DEL mft-store:cache:department:1
TTL mft-store:cache:department:1
```

در کد واقعی، JSON-B تبدیل DTO را انجام می‌دهد و Jedis دستورها را می‌فرستد. خطای عملیات Redis یا JSON-B در get باعث miss و ادامهٔ خواندن از database می‌شود.

فایل مطالعه: `organization/cache/RedisDtoCache.java` و `DepartmentDtoLoader.java`.

تمرین: چرا result.ifPresent فقط DTO موجود را ذخیره می‌کند؟ تفاوت null id و id ناموجود را در مسیر کد بررسی کنید.

۱۳ ابطال کش پس از Commit

چرا حذف فوری کافی نیست

فرض کنید transaction مقدار database را تغییر می‌دهد. اگر پیش از commit کلید حذف شود، درخواست دیگری ممکن است هنوز مقدار قدیمی را بخواند و دوباره کش کند. خود transaction نیز ممکن است rollback شود.

در پروژه، service رویداد تغییر می‌فرستد و observer پس از commit موفق کلید را حذف می‌کند.

نمونه رویداد از عملیات تغییر:

```
cacheInvalidations.fire(new CacheInvalidation(
    CacheInvalidation.Region.DEPARTMENT, id));
```

ساختار observer در RedisDtoCache به‌صورت کوتاه:

```
public void invalidate(
    @Observes(during = TransactionPhase.AFTER_SUCCESS)
    CacheInvalidation change
) {
    if (change.region() == Region.ALL_DEPARTMENTS) {
        clear(PREFIX + "department:*");
        return;
    }
    client.del(key(change.region(), change.id()));
}
```

فایل اصلی خطاهای Jedis را نیز مدیریت می‌کند؛ این مثال فقط تصمیم اصلی را نشان می‌دهد.

```
Service write --> fire event --> commit succeeds --> DEL cache
                            \-> rollback         --> no DEL
```

وابستگی DTO را فراموش نکنیم

DepartmentDto نام شعبه دارد؛ بنابراین ویرایش شعبه تمام کلیدهای دپارتمان را پاک می‌کند. اگر DTO فیلد وابستهٔ تازه بگیرد، قواعد ابطال نیز باید بررسی شوند.

تغییر عضویت کارمند کش DepartmentDto را باطل نمی‌کند؛ چون این DTO فهرست کارمندان ندارد. فهرست کارمندان از مسیر جدا خوانده می‌شود.

تمرین: ترتیب خواندن/نوشتن دو درخواست همزمان را رسم کنید. نشان دهید چرا AFTER_SUCCESS مناسب‌تر است، ولی باز هم کش کاملاً بدون دادهٔ قدیمی نمی‌سازد.

۱۴ قطعی Redis و آزمایش کش

رفتار آموزشی فعلی

خطای عملیات Redis باعث مراجعه به database می‌شود. شکست SET نباید نتیجهٔ خواندن موفق database را از بین ببرد. با این حال، تنظیم host/port نامعتبر یا خطای راه‌اندازی را با fallback همهٔ خطاها یکی ندانید.

وضعیت	انتظار
کلید موجود	بازگشت DTO کش
کلید غایب یا منقضی	خواندن database و تلاش برای SET
Redis هنگام GET قطع	خواندن database
ابطال هنگام قطعی از دست برود	مقدار قدیمی بعد از اتصال ممکن است تا TTL دیده شود
شروع برنامه	کلیدهای prefix پروژه پاک می‌شوند

پاک‌سازی startup به‌دلیل create-drop است. prefix مشترک میان چند اجرای هم‌زمان مناسب نیست؛ هر اجرا می‌تواند کش دیگری را پاک کند.

آزمایش روی دیتاست کوچک کلاس

پس از راه‌اندازی موفق برنامه و وجود یک دپارتمان واقعی:

```
docker compose exec redis redis-cli PING
docker compose exec redis redis-cli GET mft-store:cache:department:1
docker compose exec redis redis-cli TTL mft-store:cache:department:1
```

۱. GET API همان id را اجرا کنید. ۲. مقدار Redis و TTL را ببینید. ۳. GET را تکرار کنید. ۴. نام دپارتمان را از API تغییر دهید. ۵. پس از commit، نبودن کلید و سپس پرشدن مجدد را بررسی کنید.

TTL برابر -2 یعنی کلید نیست و -1 یعنی expiry ندارد؛ مقدار موجود این کش باید expiry داشته باشد.

حد سادگی

KEYS برای دیتاست کوچک مثال استفاده شده است. در دادهٔ بزرگ می‌تواند سرور را مشغول کند؛ برای سامانهٔ واقعی راهبردی مثل SCAN بررسی می‌شود. در این کلاس lock، Lua توزیع‌شده یا cache چندلایه لازم نیست.

مرجع: TTL، SET و KEYS.

۱۵ Security و پرسش‌های اصلی

دو تصمیم متفاوت

Authentication می‌پرسد «این درخواست از طرف چه کسی است؟». Authorization می‌پرسد «این شخص اجازهٔ این عملیات را دارد؟». وجود username یا هدر Authorization پاسخ خودکار این پرسش‌ها نیست.

مفهوم	مثال پروژه
Credential	username و password ارسالی
Principal	هویت تأییدشدهٔ درخواست
Role	گروهی مانند admin
Permission	نامی مانند attachment:file
Identity Store	منبع بررسی رمز و دریافت گروه‌ها
Authentication Mechanism	دریافت credential و ایجاد هویت معتبر

```
Request credentials
       |
       v
Authentication --> Principal and groups
       |
       v
Authorization --> allow endpoint OR reject
```

سیاست موردنظر Attachment

خواندن metadata عمومی است. ساخت، ویرایش و حذف metadata برای admin در نظر گرفته شده است. آپلود و دانلود محتوا گروه attachment:file می‌خواهد.

در SQL نمونه، نام permissionها همراه roleها به‌صورت group ارائه می‌شود. ردیف role/permission در database به‌تنهایی مجوز endpoint را اجرا نمی‌کند؛ identity store و قواعد دسترسی باید متصل باشند.

وضعیت فعلی را درست معرفی کنیم

BCrypt و annotationها وجود دارند، اما SecurityConfig و RolesAllowedFilter در فایل‌های فعلی نیستند؛ propertyهای اجرای security.sql نیز کامنت‌اند. امنیت کامل و ورود کاربران نمونه را آماده فرض نمی‌کنیم.

تمرین: چرا خطای «کاربر وارد شده ولی مجوز ندارد» با «هنوز وارد نشده» متفاوت است؟ کدام جزء باید تصمیم بگیرد؟

مرجع: آموزش Jakarta Security.

۱۶ نگه‌داری رمز با BCrypt

رمز را ذخیره یا برگردانیم؟

رمز خام در response، database و log نباید نگه‌داری شود. hash برای بررسی ورود ذخیره می‌شود؛ این کار با رمزنگاری قابل بازگشت فرق دارد.

BCryptPasswordHash پروژه از PasswordHash استفاده می‌کند. salt تصادفی و cost برابر 10 در generate موجودند. ستون User.password برای hash است و UserDto رمز ندارد.

استفاده از bean موجود:

```
String encoded = passwordHash.generate(
    request.password().toCharArray());

user.setPassword(encoded);
```

بررسی رمز با نمونه آموزشی:

```
boolean matches = passwordHash.verify(
    submittedPassword.toCharArray(),
    storedHash
);
```

چرا hashها را مقایسه نمی‌کنیم

یک password با saltهای متفاوت hash متفاوت می‌دهد. generate دوباره و مقایسهٔ دو رشته روش بررسی رمز نیست؛ verify را اجرا کنید.

نکته	نتیجه
salt تصادفی	hash رمز یکسان لزوماً یکسان نیست
cost	کار محاسباتی بررسی رمز
hash نامعتبر	verify موجود پروژه false می‌دهد
سقف BCrypt	72 بایت ورودی، نه 72 کاراکتر فارسی
DTO امن	hash نیز از پاسخ حذف شود

UserRequest فعلی حد طول کاراکتری دارد؛ این شرط سقف بایت UTF-8 را کنترل نمی‌کند. آموزش باید تفاوت را روشن کند، نه اینکه ورود همهٔ رمزهای چندبایتی را تضمین کند.

تمرین: برای یک رمز دو بار generate را در تست اجرا کنید. انتظار: خروجی متفاوت، ولی verify هر دو با رمز درست موفق و با رمز غلط ناموفق باشد. رمز واقعی را در log ننویسید.

فایل مطالعه: `core/security/BCryptPasswordHash.java` و تست متناظر در src/test/java.

۱۷ HTTP Basic و Jakarta Security

Basic چیست

client در هر درخواست هدر Authorization می‌فرستد:

```
Authorization: Basic <base64(username:password)>
```

Base64 رمزنگاری نیست. بیرون از آزمایش محلی، انتقال credential نیازمند HTTPS است. پاک‌کردن credential حافظهٔ UI نیز به معنی خروج از هر session یا پاک‌کردن اعتبار ذخیره‌شدهٔ مرورگر نیست.

جزء	مسئولیت
BasicAuthenticationMechanismDefinition	معرفی دریافت credential
DatabaseIdentityStoreDefinition	datasource، query رمز و query گروه‌ها
PasswordHash	تولید/بررسی hash
اتصال JNDI	اتصال به datasource واقعی
enforcement مجوز	بررسی دسترسی endpoint پس از شناخت هویت

نمونه آموزشی معرفی Basic، نه تنظیم کامل پروژه:

```
@ApplicationScoped
@BasicAuthenticationMechanismDefinition(
    realmName = "MftStore")
public class SecurityConfig {
}
```

این کلاس در کد فعلی نیست و به‌تنهایی identity store یا محدودیت endpoint را کامل نمی‌کند. lookup واقعی datasource باید با TomEE و تنظیمات پروژه تطبیق داده شود.

Query و scope

callerQuery برای hash ،username کاربر مجاز را می‌دهد؛ groupsQuery نام گروه‌ها را برمی‌گرداند. security.sql دو view به نام realm_users و realm_user_groups دارد، اما اجرای خودکارش غیرفعال است.

اگر PasswordHash سفارشی را به DatabaseIdentityStoreDefinition متصل می‌کنید، قرارداد scope آن را بررسی کنید؛ مستند Jakarta Security برای این مصرف bean با scope Dependent می‌خواهد. bean فعلی پروژه ApplicationScoped است؛ وجود آن را تنظیم نهایی login ندانید.

تمرین: وابستگی‌های لازم برای فعال‌سازی Basic را بنویسید. چرا seed کردن admin به‌تنهایی login را فعال نمی‌کند؟

مرجع: Basic mechanism و Identity Store.

۱۸ مجوزها و آزمون امنیت

annotation قرارداد را بیان می‌کند

مثال سیاست API پیوست، به‌صورت کوتاه:

```
@RolesAllowed("admin")
public class AttachmentApi {
    @GET
    @PermitAll
    public List<AttachmentDto> getAttachments() {
        return attachmentService.findAll();
    }
}
```

در مکانیزم انتخاب‌شده، annotation متد نسبت به کلاس در نظر گرفته شود. اجرای سیاست روی CDI REST باید تنظیم و با HTTP آزمایش شود؛ روی service یا UI به‌تنهایی اجراشده فرض نکنید.

ماتریس آزمون بعد از تکمیل زیرساخت

درخواست	هویت	انتظار سیاست
GET metadata	ناشناس	200
POST metadata	بدون اعتبار	401
POST metadata	معتبر، بدون admin	403
POST metadata	admin و ورودی معتبر	201
PUT content	معتبر، بدون attachment:file	403
GET content	مجاز، فایل ناموجود	404
عملیات محافظت‌شده	credential نامعتبر	رد احراز هویت

این جدول **انتظار آزمون** است، نه نتیجهٔ اجرای فعلی.

اشتباه‌های خطرناک

- مخفی‌کردن دکمهٔ حذف، امنیت endpoint نیست.
- safeJson در UI جای حذف hash از response backend نیست.
- DTO سفارش فعلی User دارد؛ دادهٔ حساس پیش از انتشار حذف شود.
- userId دلخواه client هویت تأییدشده نیست؛ عملیات حساس باید به principal معتبر مرتبط شوند.
- admin/admin123 و user/user123 در SQL فقط نمونه‌اند؛ seed فعلی اجرا نمی‌شود.

تمرین: آزمونی طراحی کنید که user عادی مستقیماً DELETE API را فراخوانی کند، حتی اگر دکمه دیده نشود.

۱۹ Attachment و جریان فایل

دو درخواست به‌جای قرارداد سنگین

پیوست ابتدا metadata و سپس محتوا می‌گیرد. body محتوا raw bytes است، نه multipart یا JSON.

نمونه POST /api/v1/attachments:

```
{
  "ownerType": "PRODUCT",
  "ownerId": 5,
  "title": "Product photo",
  "fileName": "photo.jpg",
  "description": "Sample for class"
}
```

نمونه آموزشی است؛ وجود Product شماره 5 در Attachment بررسی نمی‌شود. id پیوست ساخته‌شده از `data.id` پاسخ گرفته می‌شود.

مرحله	عملیات
ساخت metadata	POST /attachments
ارسال bytes	PUT /attachments/{id}/content
دریافت bytes	GET /attachments/{id}/content
حذف نرم metadata	DELETE /attachments/{id}

FileStorage نام UUID می‌سازد؛ نام disk با fileName ارسالی یکی نیست. Files.copy با InputStream اندازه را حساب می‌کند. پوشه بیرون از WAR است و STORE_UPLOAD_DIR آن را تعیین می‌کند.

نمونه ارسال فایل پس از گرفتن id:

```
$uploadUrl = "http://localhost:8080/api/v1/attachments/1/content"
curl.exe -X PUT $uploadUrl -H "Content-Type: image/jpeg" --data-binary "@photo.jpg"
```

بعد از فعال‌سازی امنیت، credential مجاز هم لازم است. فرمان وجود فایل یا id شماره 1 را تضمین نمی‌کند.

مرز transaction

disk بخشی از transaction database نیست. ذخیرهٔ فایل موفق با commit ناموفق می‌تواند فایل یتیم بسازد. آپلود مجدد و حذف نرم نیز فایل قبلی را پاک نمی‌کنند.

کنترل حجم، اسکن و پاک‌سازی خارج از دامنهٔ مثال‌اند. Content-Type از client می‌آید و تأیید واقعی محتوای فایل نیست.

تمرین: چرا InputStream دانلود نباید در ApiResponse قرار گیرد؟

۲۰ مستندسازی با OpenAPI

قرارداد API قابل مشاهده باشد

OpenAPI مسیر، method، پارامتر، schema و پاسخ را توصیف می‌کند. Swagger UI نمایش و آزمایش قرارداد است؛ جای اجرای service، مجوز یا validation را نمی‌گیرد.

سند attachments-openapi.json ثابت است و از annotationهای Java تولید نمی‌شود.

نمونه آموزشی ساختار سند:

```
openapi: 3.0.3
info:
  title: Attachment API
  version: "1.0"
servers:
  - url: ../api/v1
paths:
  /attachments/{id}:
    get:
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
            format: int64
      responses:
        "200":
          description: Attachment metadata
        "404":
          description: Attachment not found
```

نمونه برای فهم ساختار است؛ سند پروژه schemaهای دقیق‌تر پاسخ، خطا و محتوا دارد.

هنگام تغییر کد

مسیر/required، DTO، method و حدود طول، enum مالک، status، نوع فایل و پوشش مشترک باید هماهنگ بمانند.

server نسبی `../api/v1` نسبت به محل سند resolve می‌شود.

صفحه `/docs/index.html` و فایل `/docs/attachments-openapi.json` هستند. Swagger UI از CDN می‌آید و validator خارجی این مثال غیرفعال است.

تمرین: فرق 404 «پیوست نیست» و «فایل ندارد» را در قرارداد توضیح دهید.

مرجع: OpenAPI 3.0.3.

۲۱ UI سبک و fetch مشترک

زیرساخت کوچک به‌جای fetch تکراری

UI از همان WAR ارائه می‌شود. Bootstrap RTL ظاهر پایه را می‌دهد.

فایل	مسئولیت
core/api.js	JSON، query، request، خطا و فایل
core/ui.js	DOM، فرم و تبدیل مقدار
modules.js	منابع، فیلدها، ستون‌ها و actionها
app.js	navigation و اتصال اجزای مشترک

استفاده از API موجود، در ماژولی دارای api importشده:

```
const departments = await api.get("departments", {
    branchId: 1
});

const created = await api.post("departments", {
    branchId: 1,
    name: "Sales",
    code: "SALES",
    description: ""
});
```

get، query را در آرگومان دوم می‌گیرد. زیرساخت data را جدا می‌کند؛ صفحه نباید دوباره پوشش پاسخ را باز کند.

نمایش دادهٔ غیرقابل اعتماد

```
const cell = document.createElement("td");
cell.textContent = department.name;
```

این نمونه در context دارای department معتبر است. دادهٔ API را به شکل متن نمایش دهید؛ HTML آن را با innerHTML اجرا نکنید.

credential Basic فقط در حافظه است و فرم، login واقعی server نیست. جست‌وجو فقط روی ردیف‌های بارگذاری‌شده است. مالی readOnly است چون backend آن کامل نیست.

۲۵ منبع در ۹ گروه تعریف شده‌اند؛ وجود فرم به معنی آماده‌بودن service نیست. تغییر DTO یا مسیر باید به metadata UI هم منتقل شود.

تمرین: یک منبع جدید با columns، fields، path و paged تعریف کنید؛ زیرساخت fetch تازه نسازید.

۲۲ Docker و تنظیمات اجرا

سه سرویس محلی

Compose برنامه، PostgreSQL 18.6 و Redis را اجرا می‌کند. Dockerfile با Maven/JDK 17، WAR می‌سازد و روی TomEE 10.2.0 اجرا می‌کند.

```
Browser --> localhost:8080 --> app / TomEE
                                |
                          +------+------+
                          |             |
                       db:5432      redis:6379
```

localhost داخل container به خودش اشاره می‌کند؛ برنامه برای database از db و برای کش از redis استفاده می‌کند.

راه‌اندازی اولیه

```
Copy-Item .env.example .env
docker compose up --build -d
docker compose ps
docker compose logs --tail 100 app
```

Copy-Item فقط برای فایل محیط تازه است؛ .env موجود را بازنویسی نکنید. رمز نمونه را تغییر دهید. Engine باید روشن باشد؛ نبودن STORE_DB_PASSWORD خطای Compose می‌دهد.

تنظیم	کاربرد
STORE_DB_URL / USER / PASSWORD	اتصال برنامه
STORE_DB_NAME	نام database در Compose
STORE_REDIS_HOST / PORT	پیش‌فرض محلی localhost:6379
STORE_UPLOAD_DIR	مسیر فایل؛ داخل Compose /data/uploads
POSTGRES_PORT / REDIS_PORT / APP_PORT	پورت میزبان، نه پورت داخلی

هشدار داده

Hibernate فعلاً create-drop دارد. volume، حذف جدول توسط Hibernate را متوقف نمی‌کند. فایل‌ها volume جدا دارند؛ Redis موقت است.

volume، docker compose down را نگه می‌دارد. گزینه -v، volume و دادهٔ آن را حذف می‌کند؛ بدون نیاز آن را اجرا نکنید.

Healthcheck موجود برای db و Redis است؛ سلامت آن‌ها deploy سالم app را ثابت نمی‌کند.

۲۳ تست و تشخیص خطا

هر تست چه چیزی را می‌سنجد

نوع	بررسی	تأیید نمی‌کند
entity/repository با H2	mapping و query	همه تفاوت‌های PostgreSQL
mapper مستقیم	تبدیل و حفظ فیلد	Lazy یا CDI سرور
service با Mockito	قواعد و فراخوانی	commit واقعی JTA
REST مستقیم	DTO و status متد	filter و authentication واقعی
storage با پوشه موقت	bytes و فایل	مجوز HTTP و حجم
HTTP روی TomEE	integration	همه سناریوهای هم‌زمانی

نتیجه ثبت‌شده در ۲۰۲۶/۱۰/۱۳

۵۴ تست اجرا شد: ۳۰ Attachment و ۶ Core موفق؛ ۱۸ تست entity انبار ناموفق، با ۲ failure و ۱۶ error. testPU فقط Attachment را ثبت کرده و exclude-unlisted-classes=true دارد.

```
.\mvnw.cmd -B -ntp test
.\mvnw.cmd -B -ntp -DskipTests package
```

فرمان دوم WAR ساخت، ولی تست را اجرا نکرد. این نتیجه deploy، کش، OpenAPI یا browser را تأیید نمی‌کند. هشدار مقدار اولیهٔ Delivery.status در builder نیز باقی است.

ترتیب عیب‌یابی

۱. **Compile:** import، processor و کد تولیدشده.
۲. **Deploy:** خطای CDI/JNDI/datasource در log سرور.
۳. **400:** DTO و validation.
۴. **404:** مسیر و id.
۵. **409:** قاعدهٔ business.
۶. **500:** Lazy، stack trace و متدهای stub.

برای entity تازه، وابستگی‌هایش نیز در persistence تست ثبت شوند. وجود فایل تست به معنی وجود method دارای Test نیست.

تمرین: چرا ثبت‌نشدن Inventory، اثبات خرابی محاسبهٔ موجودی نیست؟

۲۴ کارگاه ساخت یک ماژول تمیز

ترتیب پیشنهادی تحویل

مرحله	خروجی
۱ مدل	entity، رابطه، ستون و enum
۲ داده	repository و finder
۳ قرارداد	Request و DTO بدون entity
۴ تبدیل	mapper با mapping رابطه
۵ business	transaction، service و exception
۶ API	paging، CRUD و عملیات ویژه
۷ آزمون	حالت درست، validation، نبودن و تعارض
۸ اتصال	metadata UI و مستند قرارداد

سناریوی تمرین دپارتمان

برای شعبهٔ موجود، نام و کد معتبر بفرستید. همان کد در همان شعبه را دوباره بفرستید؛ انتظار BusinessException داشته باشید. سپس id ناموجود را بخوانید و 404 را با خطای ورودی اشتباه نگیرید.

branchId، Request می‌گیرد. service شعبهٔ واقعی را می‌خواند؛ branch/status/employees mapper را از ورودی نمی‌گیرد. وضعیت action جدا و کارمند مسیر membership دارد.

مواردی که باید از تحویل حذف شوند

- متد نمایشی که همیشه null یا Optional خالی می‌دهد.
- entity یا password/hash در پاسخ JSON.
- محاسبهٔ مبلغ یا فیلد server-owned از client.
- query با چسباندن ورودی کاربر.
- catchهای تکراری صرفاً برای پاسخ‌سازی در REST.
- endpoint که UI ناموجود را آماده معرفی می‌کند.
- annotation امنیت بدون بررسی اجرای واقعی.

معیار پایان

Build، تست‌های اصلی، deploy سالم و یک دور API واقعی لازم‌اند. برای کش، commit/rollback، hit/miss و قطعی بررسی شود.

framework فرانت‌اند، JWT، کش چندلایه و generic framework تازه لازم نیست؛ service خوانا و قرارداد روشن برای این درس مناسب‌تر است.

تمرین پایانی: «ویرایش دپارتمان و خواندن مجدد از Redis» را از UI تا observer commit توضیح دهید.

۲۵ پرسش‌های مرور و پاسخ

ابتدا بدون نگاه به پاسخ جواب دهید

پرسش	پاسخ
چرا entity در DTO نباشد؟	Lazy، رابطه، حلقه JSON و دادهٔ حساس ممکن است منتشر شود
mapper مسئول بارگذاری Branch است؟	نه؛ service آن را می‌خواند و قواعد را اجرا می‌کند
build موفق یعنی mapping کامل؟	نه؛ IGNORE می‌تواند فیلد بدون mapping را پنهان کند
MappingTarget چه می‌کند؟	تغییر entity موجود، نه ساخت نمونهٔ تازه
read همیشه بدون transaction است؟	نه؛ تبدیل رابطهٔ Lazy ممکن است context بخواهد
چرا DTO کش شود؟	داده ساده، بدون proxy و وابستگی JPA
TTL به‌تنهایی کافی است؟	نه؛ update ابطال می‌خواهد و race ممکن است
چرا AFTER_SUCCESS؟	ابطال وابسته به commit موفق است
Basic رمزنگاری است؟	نه؛ encoding است و انتقال امن لازم است
hashها را مقایسه کنیم؟	نه؛ salt متفاوت است و verify لازم است
RolesAllowed یعنی امنیت فعال؟	نه؛ mechanism و enforcement باید بررسی شوند
volume مانع create-drop است؟	نه؛ Hibernate جدول داخل database را حذف می‌کند

تمرین‌های تشریحی

الف: Redis قطع است و update موفق می‌شود. پس از اتصال چه داده‌ای ممکن است خوانده شود؟
پاسخ: اگر ابطال از دست برود، کلید قدیمی ممکن است تا پایان expiry دیده شود.

ب: mapper، branch.name می‌خواند ولی context بسته است.
پاسخ: خواندن و تبدیل DTO داخل مرز transactional معتبر انجام شود؛ ignore کردن relation پاسخ درست نمی‌سازد.

ج: دکمهٔ حذف برای user مخفی است.
پاسخ: server باید مجوز endpoint را بررسی کند؛ client قابل اعتماد نیست.

د: quantity با مجموع گردش‌ها متفاوت است.
پاسخ: ورودی، تغییر مستقیم quantity، transaction اتمیک و تداخل همزمان بررسی شود؛ فقط UI را اصلاح نکنید.

۲۶ واژه‌نامه و منابع مطالعه

واژه	معنی
Container	محیط اجرای مدیریت‌شده
Bean / CDI	شیء مدیریت‌شده و تزریق وابستگی
Entity / DTO	مدل پایدار / قرارداد تبادل
Persistence Context	مجموعهٔ entityهای managed
Commit / Rollback	نهایی‌کردن / برگرداندن تغییر
Dirty Checking	تشخیص تغییر entity managed
Lazy	بارگذاری رابطه هنگام نیاز
Hit / Miss	یافتن / نیافتن داده در کش
Principal / Role	هویت معتبر / گروه دسترسی
WAR	بستهٔ deploy برنامهٔ وب

فایل‌های مطالعه در پروژه

مسیرها نسبت به package اصلی در src/main/java هستند:

- `core/entity/BaseEntity.java` و `core/repository/BaseRepository.java`
- `core/mapper/BaseMapper.java` و `MapStructConfig.java`
- `organization/mapper/DepartmentMapper.java`
- `organization/cache/RedisDtoCache.java` و `DepartmentDtoLoader.java`
- `core/security/BCryptPasswordHash.java`
- `attachments/rest/AttachmentApi.java` و `attachments/storage/FileStorage.java`

منابع رسمی

۱. Jakarta EE 10 و CDI 4.0
۲. Persistence 3.1 و Transactions 2.0
۳. MapStruct 1.6
۴. Redis Cache Aside و KEYS
۵. Security 3.0 و آموزش Security
۶. OpenAPI 3.0.3

متن، تمرین و شرح مثال‌های پروژه برای این جزوه تدوین شده‌اند؛ منابع رسمی برای مطالعهٔ بیشترند. نمونهٔ آموزشی جای تکمیل و آزمون کد را نمی‌گیرد.

تهیه و تدوین احمد مصباح